

GLA UNIVERSITY , MATHURA

OPERATING SYSTEM LAB

B.TECH SEMESTER III

(Code: BCSC 0803)

Name: Vivek Kashyap.

University Roll no.: 2415001808.

Class Roll no.: 68.

Section: 2SC.

Submitted to: Mr. Sumit Garg.

Faculty Signature: . . . . . .

Operating Systems Lab (BCSC0803)

Assignment 1: Implement Basic UNIX/Linux Commands

Topic: *Implement the following basic commands (with options) used in UNIX/LINUX OS*
Marks: 10
Course: Operating Systems Lab (BCSC0803)
Objective:
To familiarize students with the Linux command-line environment and develop the ability to manage files, directories, and processes using fundamental UNIX commands.

Assignment Scenario

You are newly appointed as a **Junior System Administrator** at a small startup called **TechZen Pvt. Ltd.**
Your task is to organize the company's user workspace on a Linux system and perform several operations such as creating directories, managing files, setting permissions, and viewing system information.

You must use only **UNIX/Linux commands** (no GUI operations) to accomplish the following tasks.

Tasks to Perform (Write All Commands and Their Outputs)

Part A: Directory & File Management (4 Marks)

1. Create a main directory named techzen_admin in your home directory.
2. Inside it, create three subdirectories:
    - projects
    - employees
    - reports
3. Navigate inside projects and create three empty text files named alpha.txt, beta.txt, and gamma.txt.
4. Copy all files from projects to the reports folder.
5. Move beta.txt to the employees folder and rename it to empdata.txt.
6. Delete the gamma.txt file from the projects folder.

CODE:

1. **mkdir ~/techzen_admin: Creates a directory named techzen_admin in the home folder.**

2. cd ~/techzen_admin

mkdir projects employees reports: Creates three subfolders inside techzen_admin.

3. cd projects

touch alpha.txt beta.txt gamma.txt: Creates three empty text files inside projects.

4. cp *.txt ../reports/: Copies all files from projects to reports.

5. mv beta.txt ../employees/empdata.txt: Moves beta.txt to employees and renames it to empdata.txt.

6. rm gamma.txt: Deletes gamma.txt from projects.

SCREENSHOT:

```
~$ mkdir ~/techzen_admin
~$ cd ~/techzen_admin
~/techzen_admin$ mkdir projects employees reports
~/techzen_admin$ cd projects
~/techzen_admin/projects$ touch alpha.txt beta.txt gamma.txt
~/techzen_admin/projects$ cp *.txt ../reports/
~/techzen_admin/projects$ mv beta.txt ../employees/empdata.txt
~/techzen_admin/projects$ rm gamma.txt
```

Part B: Viewing and Searching (2 Marks)

1. Display the list of all files and directories with detailed information (size, date, permissions, etc.).
   (Hint: use ls -l)
2. Display only the first 5 lines and last 3 lines of /etc/passwd.
3. Search for the word bash in /etc/passwd and count how many users use bash shell.
   (Hint: use grep and wc)

CODE:

1. **ls -l ~/techzen_admin: Shows permissions, owner, size, and date of files.**
2. **head -5 /etc/passwd & tail -3 /etc/passwd: Shows first five and last three lines of the passwd file.**
3. **grep bash /etc/passwd | wc -l**

- **Counts how many users are using the bash shell.**

SCREENSHOT:

```
~/techzen_admin/projects$ ls -l ~/techzen_admin
total 2
drwxr-xr-x 2 user user 3 Nov  9 17:49 employees
drwxr-xr-x 2 user user 3 Nov  9 17:49 projects
drwxr-xr-x 2 user user 5 Nov  9 17:49 reports
~/techzen_admin/projects$ head -5 /etc/passwd
root:x:0:0:root:/root:/bin/bash
daemon:x:1:1:daemon:/usr/sbin:/usr/sbin/nologin
bin:x:2:2:bin:/bin:/usr/sbin/nologin
sys:x:3:3:sys:/dev:/usr/sbin/nologin
sync:x:4:65534:sync:/bin:/bin/sync
~/techzen_admin/projects$ tail -3 /etc/passwd
_rpc:x:116:65534::/run/rpcbind:/usr/sbin/nologin
tcpdump:x:117:126::/nonexistent:/usr/sbin/nologin
rstudio-server:x:997:987::/home/rstudio-server:/bin/sh
~/techzen_admin/projects$ grep bash /etc/passwd | wc -l
5
```

Part C: File Permissions (2 Marks)

1. Display the current permissions of all files inside techzen_admin.
2. Change the permission of alpha.txt to allow the owner to read, write, and execute, the group to read and execute, and others to only read.
   (Hint: chmod 754 alpha.txt)
3. Verify that the permissions have changed successfully using ls -l.

CODE:

1. **ls -l ~/techzen_admin/projects: Shows existing permissions of files.**
2. **chmod 754 ~/techzen_admin/projects/alpha.txt: Gives owner full rights, group read/execute, others read only.**
3. **3.ls -l ~/techzen_admin/projects: Checks the updated permission.**

SCREENSHOT:

```
~/techzen_admin/projects$ ls -l ~/techzen_admin/projects
total 1
-rw-r--r-- 1 user user 0 Nov  9 17:49 alpha.txt
~/techzen_admin/projects$ chmod 754 ~/techzen_admin/projects/alpha.txt
~/techzen_admin/projects$ ls -l ~/techzen_admin/projects
total 1
-rwxr-xr-- 1 user user 0 Nov  9 17:49 alpha.txt
```

Part D: System Information & Process Commands (2 Marks)

1. Display the current date and time.
2. Show the current working directory.
3. Display the list of all running processes.

4. Create a background process using any command (e.g., gedit & or sleep 100 &) and terminate it using the kill command.

5. SCREENSHOT:

```
~/techzen_admin/projects$ date
Sun Nov  9 18:18:58 UTC 2025
~/techzen_admin/projects$ pwd
/home/user/techzen_admin/projects
~/techzen_admin/projects$ ps
    PID TTY          TIME CMD
    868 pts/0    00:00:00 bash
   1495 pts/0    00:00:00 ps
~/techzen_admin/projects$ sleep 100 &
[1] 1496
~/techzen_admin/projects$ ps
    PID TTY          TIME CMD
    868 pts/0    00:00:00 bash
   1496 pts/0    00:00:00 sleep
   1497 pts/0    00:00:00 ps
~/techzen_admin/projects$ kill <process_id>
bash: syntax error near unexpected token `newline'
```

CODE:

1. **Date: Shows system date and time.**
2. **pwd: Displays current path.**
3. **ps: Lists currently running processes.**
4. **sleep 100 &: ps kill <process_id>**

- **Creates a sleep process in background and kills it using its process ID**

6. CONCLUSION:

From this practical, I learned how to manage files and directories using Linux commands.
I also practiced setting permissions, searching within files, and managing system processes.
This helped me understand how system administrators organize and maintain user workspaces efficiently.

Assignment 2 – Write and Implement Basic vi Editor Commands

Topic: Use the vi editor to create, edit, save and manipulate text files.
Marks: 10
Course: Operating Systems Lab (BCSC0803)

Objective

To learn how to:

- open, edit and save files in vi
- switch between modes

- perform navigation, copy, paste and delete operations
- use search and replace commands

Assignment Scenario

You are working as a **Junior Linux Administrator** at *CodeWave Solutions*.
The company keeps daily configuration logs on a Linux server.Your task is to use the **vi editor** to create and manage today's log file, following the exact instructions below.

Tasks to Perform (Write Commands and Explain Output)

Part A – Create and Edit File (3 marks)

1. Open a new file called system_log.txt using vi.
2. vi system_log.txt
3. Enter *Insert mode* and type the following text (or your own 4–5 lines):
    - **CodeWave Solutions – System Log**
    - **Server check completed successfully.**
    - **Backup started at 9:00 AM.**
    - **No critical errors detected.**
    - **Backup finished at 9:20 AM.**
    - **Save and exit using : wq**

```
~$ vi system_log.txt
~$ cat system_log.txt
CodeWave Solutions – System Log
Server check completed successfully.
Backup started at 9:00 AM.
No critical errors detected.
Backup finished at 9:20 AM.
```

Part B – Navigation and Editing (3 marks)

1. Re-open the same file.
2. Move between lines using j (down) and k (up).
3. Delete the third line using dd.
4. Copy the first line (yy) and paste it below the last line (p).

5. Undo the last action (u) and redo it (Ctrl + r).

```
CodeWave Solutions – System Log
Server check completed successfully.
Backup started at 9:00 AM.
No critical errors detected.
Backup finished at 9:20 AM.
j   → move down one line
k   → move up one line
CodeWave Solutions – System Log
```

Part C – Search and Replace (2 marks)

1. Search for the word Backup using /Backup and press n to jump to the next match.
2. Replace the word Backup with Data Backup throughout the file:

   :%s/Backup/Data Backup/g

```
~$ vi system_log.txt
~$ cat system_log.txt
CodeWave Solutions – System Log
Server check completed successfully.
Data Data Backup started at 9:00 AM.
No critical errors detected.
j   → move down one line
Log verified by admin
k   → move up one line
CodeWave Solutions – System Log
```

Part D – Display and Line Numbers (2 marks)

1. Show line numbers: set number
2. Move to a specific line (e.g., line 5): 5
3. Append a new line below using o and type "Log verified by admin."
4. Save and exit using: x

```
CodeWave Solutions – System Log
Server check completed successfully.
Data Data Backup started at 9:00 AM.
No critical errors detected.
j   → move down one line
Log verified by admin
k   → move up one line
CodeWave Solutions – System Log
~
```

Assignment 3 – Shell Scripts Using Simple Commands

Course: Operating Systems Lab (BCSC0803)

Topic: Write and execute shell scripts using simple UNIX/Linux commands

Marks: 10

Objective

To learn how to:
- Create and run simple shell scripts in Linux
- Use variables, echo statements, and basic command substitution
- Combine basic Linux commands inside shell scripts

Scenario

You have recently joined TechNova Pvt. Ltd. as a Linux System Trainee. Your task is to automate routine system checks using simple shell scripts. Your manager has asked you to create three small shell scripts to perform basic system operations such as displaying user details, file information, and arithmetic operations.

Tasks to Perform

Part A – Display System and User Information (3 Marks)

File Name: sysinfo.sh

Instructions:
1. Create a script that displays:
   - Current username
   - Current date and time
   - Current working directory
   - List of files in current directory
2. Use suitable Linux commands (whoami, date, pwd, ls).

Expected Output Example:

User Name: student
Date and Time: Mon Oct 13 11:40:21 IST 2025
Current Directory: /home/student/os_lab

Files:
alpha.txt beta.txt system_log.txt

```
 GNU nano 7.2
echo "User Name: $(whoami)"
echo "Date and Time: $(date)"
echo "Current Directory: $(pwd)"
echo "Files:"
ls
```

```
~$ chmod +x display.sh
~$ ./display.sh
User Name: user
Date and Time: Sun Nov  9 13:59:35 UTC 2025
Current Directory: /home/user
Files:
2025-11-03-terminal-2.term  OS          assignment5  check.sh     demo        file2      file6.txt       raj
6                           O_S         bh           compare.sh   display.sh  file3      looping.sh      substring.sh
Dir                         addition.sh bhawana      compare2.sh  file.sh     file4.txt  myfirstfile.sh
MYFIRSTFILE.term            all4.sh     case.sh      concat.sh    file1       file5.txt  operation.sh
```

Explanation: This script displays the current username using whoami, shows the system date and time using date, and prints the current working directory using pwd.
It then lists all files in that directory using ls.
It's used to show basic system and user information.

Part B – File and Directory Operations (3 Marks)

File Name: fileops.sh

Instructions:

1. Create a script that:
   - Asks the user for a folder name
   - Creates the folder
   - Navigates into it
   - Creates three text files
   - Displays the list of created files
2. Use commands: mkdir, cd, touch, ls, echo.

Expected Output Example:

Enter folder name: demo
Folder created successfully.
Files created:
file1.txt file2.txt file3.txt

```
  GNU nano 7.2
echo -n "Enter folder name: "
read folder
mkdir "$folder"
echo "Folder created successfully."
cd "$folder"
touch file1.txt file2.txt file3.txt
echo "Files created:"
ls
```

```
~$ chmod +x file.sh
~$ ./file.sh
Enter folder name: student
Folder created successfully.
Files created:
file1.txt  file2.txt  file3.txt
```

Explanation: This script asks for a folder name, creates that folder using mkdir, and navigates into it with cd.
Then it creates three text files using touch.
Finally, it lists the created files using ls.

Part C – Simple Arithmetic Operations (4 Marks)

File Name: calc.sh

Instructions:
1. Create a script that:
   - Asks the user to enter two numbers
   - Performs addition, subtraction, multiplication, and division
   - Displays the results clearly
2. Use read, expr, or $(( )) for arithmetic operations.

Expected Output Example:

Enter first number: 10
Enter second number: 5
Addition: 15
Subtraction: 5
Multiplication: 50
Division: 2

```
 GNU nano 7.2
echo -n "Enter first number: "
read num1
echo -n "Enter second number: "
read num2
add=$((num1 + num2))
sub=$((num1 - num2))
mul=$((num1 * num2))
div=$((num1 / num2))
echo "Addition: $add"
echo "Subtraction: $sub"
echo "Multiplication: $mul"
echo "Division: $div"
```

```
~$ chmod +x operation.sh
~$ ./operation.sh
Enter first number: 10
Enter second number: 5
Addition: 15
Subtraction: 5
Multiplication: 50
Division: 2
```

Explanation: This shell script takes two numbers as input using read, performs basic arithmetic operations (addition, subtraction, multiplication, and division) using $(( )), and stores results in variables.
Then it prints each result using echo.
It's a simple calculator program in bash.

Assignment 4 – Shell Scripts using Conditional, Looping, and Case Statements

Course: Operating Systems Lab (BCSC0803)
Topic: Shell Scripts using Conditional, Looping, and Case Statements
Marks: 10

Objective

To understand and implement shell scripts that use conditional, looping, and case statements for decision-making and repetitive operations in Linux.

Scenario

You are working as a **Junior Automation Engineer** at *TechSolutions Pvt. Ltd.*
Your senior has asked you to write a few scripts to automate common system tasks using **decision-making** and **loop control statements**.
Each task requires the use of *if-else conditions*, *loops*, or *case statements*.

Tasks to Perform

Part A – Conditional Statements (3 Marks)

File Name: condition_check.sh

Instructions:

1. Write a shell script that:
    - Accepts a number from the user.
    - Checks whether the number is **positive**, **negative**, or **zero** using if, elif, and else.
2. Display the result clearly.

Expected Output Example:

Enter a number: -3

The number is Negative

CODE:

```
echo -n "Enter a number: "

read num

if [ $num -gt 0 ]

then
```

```
    echo "The number is Positive"
elif [ $num -lt 0 ]
then
    echo "The number is Negative"
else
    echo "The number is Zero"
fi
```

OUTPUT:

```
~$ vi condition_check.sh
~$ chmod +x condition_check.sh
~$ ./condition_check.sh
Enter a number: -3
The number is Negative
```

Explanation:

- It first reads the input into the variable `num`.
- If `num` is greater than 0**,** it prints "The number is Positive."
- If `num` is less than 0, it prints "The number is Negative."
- Otherwise, it prints "The number is Zero."

In short: it simply tells you whether the number you entered is positive, negative, or zero.

Part B – Looping Statements (4 Marks)

File Name: loop_practice.sh

Instructions:

1. Write a shell script that:
    - Displays a **menu** with the following options:
        1. Display numbers 1 to 10 using for loop
        2. Display even numbers between 1 to 20 using while loop
        3. Display countdown from 5 to 1 using until loop
        4. Exit
    - Execute the selected option accordingly.

Expected Output Example:

MENU

1. Numbers 1 to 10

2. Even numbers 1 to 20

3. Countdown 5 to 1

4. Exit

Enter your choice: 2

Even Numbers: 2 4 6 8 10 12 14 16 18 20

CODE:

```
#!/bin/bash
while true
do
   echo "MENU"
   echo "1. Numbers 1 to 10"
   echo "2. Even numbers 1 to 20"
   echo "3. Countdown 5 to 1"
   echo "4. Exit"
   echo -n "Enter your choice: "
   read choice
   case $choice in
      1)
         echo -n "Numbers: "
         for i in {1..10}
         do
            echo -n "$i "
         done
         echo ""
         ;;
      2)
         echo -n "Even Numbers: "
         i=2
         while [ $i -le 20 ]
         do
            echo -n "$i "
```

```
            i=$((i+2))
        done
        echo ""
        ;;
    3)
        echo -n "Countdown: "
        i=5
        until [ $i -lt 1 ]
        do
            echo -n "$i "
            i=$((i-1))
        done
        echo ""
        ;;
    4)
        echo "Exiting..."
        break
        ;;
    *)
        echo "Invalid choice! Please try again."
        ;;
  esac
  echo ""
done
```

OUTPUT:

```
~$ vi loop_practice.sh
~$ chmod +x loop_practice.sh
~$ ./loop_practice.sh
MENU
1. Numbers 1 to 10
2. Even numbers 1 to 20
3. Countdown 5 to 1
4. Exit
Enter your choice: 2
Even Numbers: 2 4 6 8 10 12 14 16 18 20
```

EXPLANATION:

- If you select 1, it uses a for loop to display numbers from 1 to 10.
- If you select 2, it uses a while loop to print even numbers between 1 and 20**.**
- If you select 3**,** it uses an until loop to show a countdown from 5 to 1**.**
- If you select 4, it prints "Exiting..." and stops the program.
- If you enter anything else, it shows "Invalid choice!"

In short: the script demonstrates for**,** while, and until loops inside a menu-driven program that runs repeatedly until you exit.

Part C – Case Statement (3 Marks)

File Name: case_menu.sh

Instructions:

1. Write a shell script that:
    - Displays a menu with the following options:
      a. Show current date and time
      b. Display current logged-in user
      c. Show current directory path
      d. Exit
    - Use a **case statement** to execute based on user input.

Expected Output Example:

MENU

a. Date and Time

b. Logged-in User

c. Current Directory

d. Exit

Enter your choice: b

Current User: student

CODE:

```
#!/bin/bash

echo "MENU"

echo "a. Date and Time"

echo "b. Logged-in User"

echo "c. Current Directory"

echo "d. Exit"
```

```
echo -n "Enter your choice: "
read choice
case $choice in
  a)
    echo "Current Date and Time: $(date)"
    ;;
  b)
    echo "Current User: $(whoami)"
    ;;
  c)
    echo "Current Directory: $(pwd)"
    ;;
  d)
    echo "Exiting..."
    ;;
  *)
    echo "Invalid option! Please choose a, b, c, or d."
    ;;
esac
```

OUTPUT:

```
~$ vi case_menu.sh
~$ chmod +x case_menu.sh
~$ ./case_menu.sh
MENU
a. Date and Time
b. Logged-in User
c. Current Directory
d. Exit
Enter your choice: b
Current User: user
```

EXPLANATION:

This shell script displays a menu with four options and performs an action based on what you choose.

- If you press a, it shows the current date and time.
- If you press b, it displays the logged-in user's name.

- If you press c, it shows the current working directory.
- If you press d, it prints "Exiting..." and stops.
- If you enter anything else, it shows "Invalid option!"

In short, it is a simple menu-based program that uses a case statement to run different commands based on user input.

Assignment 5 – Shell Scripts Related to Strings

Course: Operating Systems Lab (BCSC0803)
Topic: Shell Scripts Related to Strings
Marks: 10

Objective

To learn how to perform basic string operations in shell scripting such as string creation, concatenation, comparison, length calculation, substring extraction, and reversal.

Scenario

You are working as a **System Automation Intern** at *TechnoSoft Pvt. Ltd.*
Your mentor has assigned you a few small scripting tasks that involve **string processing**.
You are required to write shell scripts that can handle and manipulate user-entered strings efficiently.

Tasks to Perform

Part A – String Comparison (3 Marks)

File Name: string_compare.sh

Instructions:

1. Write a shell script to:
    - Accept two strings from the user.
    - Compare them using the = and != operators.
    - Display whether both strings are equal or not.

Expected Output:

Enter first string: Linux

Enter second string: Unix

Strings are different.

CODE :

```
echo -n "enter string 1: "

read string1

echo -n "enter string2: "
```

```
read string2
if [ "$string1" = "$string2" ];
then
    echo "Strings are equal":
else
    echo "Strings are different";
fi
```

OUTPUT:

```
~/topic5$ vi string_ops.sh
~/topic5$ chmod +x string_ops.sh
~/topic5$ ./string_ops.sh
enter string 1: Linux
enter string2: Unix
Strings are different
```

EXPLANATION:

The script reads two strings and compares them. If both are same, it prints "Strings are equal" otherwise, *"Strings are different"*

Part B – String Concatenation and Length (4 Marks)

File Name: string_concat_length.sh

Instructions:

1. Write a shell script to:
    - Read two strings from the user.
    - Concatenate them with a space in between.
    - Display the concatenated string.
    - Calculate and display the length of both original strings and the final concatenated string.

Expected Output:

Enter first string: Operating

Enter second string: System

Concatenated String: Operating System

Length of first string: 9

Length of second string: 6

Length of concatenated string: 16

CODE:

```
echo -n "enter str1: "
read str1
echo -n "enter str2: "
read str2
result="$str1 $str2"
echo "result : $result"
length1=${#str1}
length2=${#str2}
length3=${#result}
echo "Length of str1 ($str1) is: $length1"
echo "Length of str2 ($str2) is: $length2"
echo "Length of concatenated string ($result) is: $length3"
```

OUTPUT:

```
~/topic5$ vi string_concat_length.sh
~/topic5$ chmod +x string_concat_length.sh
~/topic5$ ./string_concat_length.sh
enter str1: Operating
enter str2: System
result : Operating System
Length of str1 (Operating) is: 9
Length of str2 (System) is: 6
Length of concatenated string (Operating System) is: 16
```

EXPLANATION:

The script joins both strings with a space and prints individual and total lengths.

Part C – Substring Extraction and Reversal (3 Marks)

File Name: substring_reverse.sh

Instructions:

1. Write a shell script to:
    - Accept a string from the user.
    - Extract the first 5 characters of the string.
    - Reverse the entire string and display it.

Expected Output:

Enter a string: ComputerScience

First 5 characters: Compu

Reversed string: ecneicSretupmoC

CODE:

```
echo -n "Enter a str"

read str

first_five=${str:0:5}

reversed=$(echo "$str" | rev)

echo "First 5 characters: $first_five"

echo "Reversed string: $reversed"
```

OUTPUT:

```
~/topic5$ vi substring_reverse.sh
~/topic5$ chmod +x substring_reverse.sh
~/topic5$ ./substring_reverse.sh
Enter a strComputerScience
First 5 characters: Compu
Reversed string: ecneicSretupmoC
```

EXPLANATION:

The script takes a string, extracts the first five characters, then reverses the full string using the rev command.

Assignment 6 – Shell Scripts Using Pipes

Course: Operating Systems Lab (BCSC0803)
Topic: Shell Scripts Using Pipes
Marks: 10

Objective

To understand how to use the pipe (|) operator in shell scripting for connecting multiple commands, allowing the output of one command to be used as input for another command.

Scenario

You are working as a **Junior Linux Administrator** at *DataPro Systems*.
Your supervisor has assigned you tasks that involve analyzing files and user data.
You are expected to write **efficient shell scripts** using **pipes** to combine commands for automating daily tasks like filtering, counting, and sorting.

Tasks to Perform

Part A – File and User Information (3 Marks)

File Name: pipe_userinfo.sh

Instructions:

1. Display the **total number of users** on the system.
2. Display the **total number of files** in the current directory.
3. Display the **list of logged-in users** in alphabetical order.

```
~/Himanshu$ nano pipe_userinfo.sh
~/Himanshu$ chmod +x pipe_userinfo.sh
~/Himanshu$ ./pipe_userinfo.sh
./pipe_userinfo.sh: line 3: Count: command not found
Total users: 46
./pipe_userinfo.sh: line 7: Count: command not found
Total files in current directory: 2
./pipe_userinfo.sh: line 11: Logged-in: command not found
Logged-in users:
~/Himanshu$ bash pipe_userinfo.sh
pipe_userinfo.sh: line 3: Count: command not found
Total users: 46
pipe_userinfo.sh: line 7: Count: command not found
Total files in current directory: 2
pipe_userinfo.sh: line 11: Logged-in: command not found
Logged-in users:
```

Part B – Text Data Processing (4 Marks)

File Name: pipe_textops.sh

Instructions:

1. Ask the user for a filename and a keyword.
2. Display the total number of lines containing the keyword.
3. Display all unique words in that file in sorted order.
4. Display the total word count of the file.

Hints:

- Use grep, sort, uniq, and wc with pipes.
- Example pipeline: cat file.txt | grep keyword | wc -l

Expected Output:

```
~/Himanshu$ nano pipe_tesxttops.sh
~/Himanshu$ chmod +x pipe_tesxttops.sh
~/Himanshu$ nano data.txt
~/Himanshu$ chmod +x data.txt
~/Himanshu$ ./pipe_tesxttops.sh
Enter filename:
data.txt
Enter keyword:
Linux
./pipe_tesxttops.sh: line 9: Count: command not found
Lines containing keyword: 1
./pipe_tesxttops.sh: line 13: Unique: command not found
Unique sorted words:
Data
Linux
Network
Server
System
./pipe_tesxttops.sh: line 17: Total: command not found
Total words: 8
```

Part C – Log File Filtering (3 Marks)

File Name: pipe_logfilter.sh

Instructions:

1. Ask the user for a log filename.
2. Display only the last 10 lines containing the word “error”.
3. Count how many such lines exist in total.

Hints:

- Use grep, wc, and tail with pipes.

Expected Output:

```
~/Himanshu$ nano pipe_logfilter.sh
~/Himanshu$ chmod +x pipe_logfilter.sh
~/Himanshu$ nano system.log
~/Himanshu$ ./pipe_logfilter.sh
Enter log file name:
system.log
./pipe_logfilter.sh: line 6: Check: command not found
Last 10 lines with 'error':
[Error] Permission denied
[Error] Failed to load module
[Error] Authentication failed
[Error] Database connection lost
[Error] File system corrupted
[Error] Unexpected shutdown
[Error] Service crash detected
[Error] Kernel panic detected
[Error] Backup failed
[Error] Timeout while accessing server
Total errors found: 13
~/Himanshu$
```